上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

2026年春季学期

编译原理
Principles of Compilers

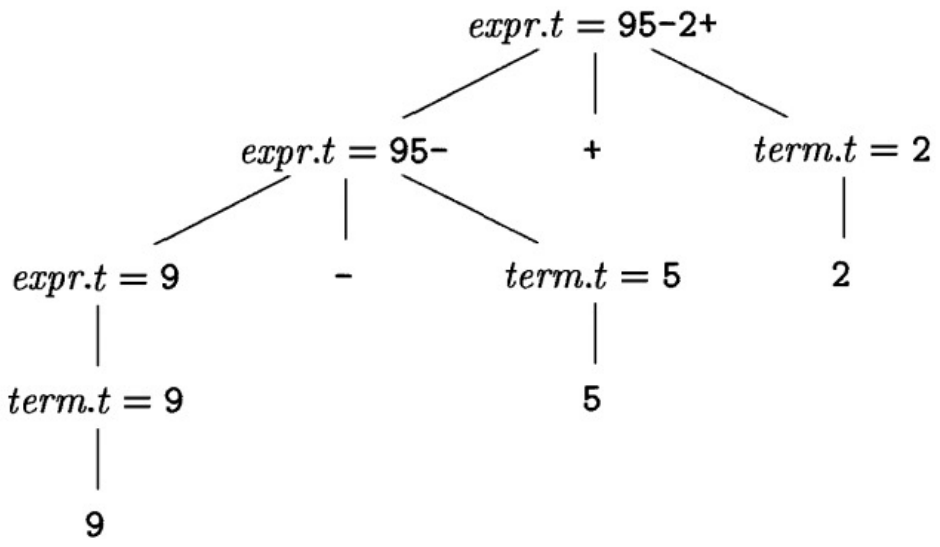
第三章：词法分析（1）

赵杰茹

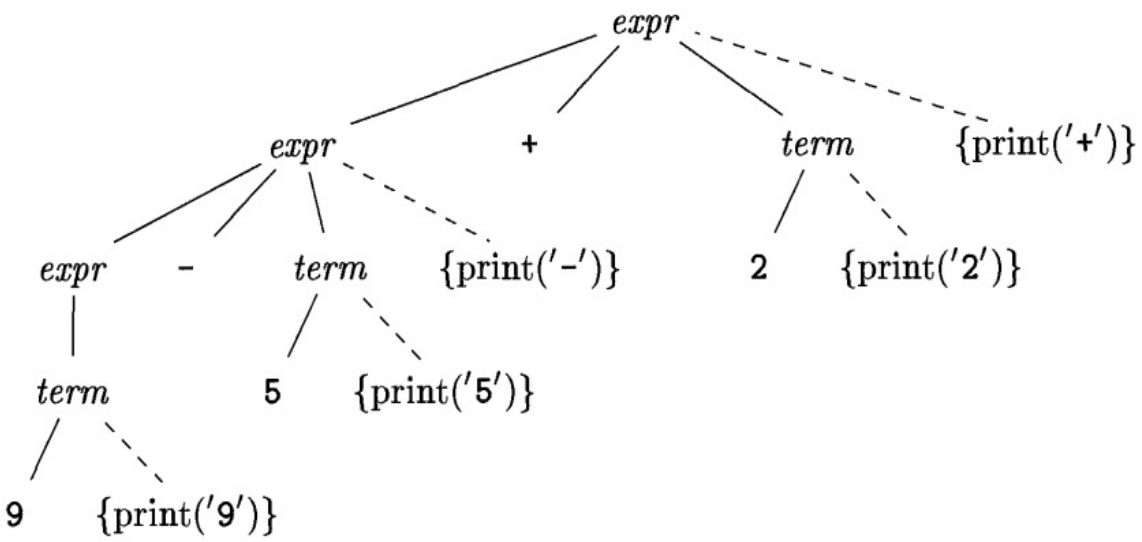
上海交通大学计算机学院

回顾：

- 语法制导定义将字符串作为属性附加到语法分析树的各结点上
- 语法制导翻译方案通过语义动作，将翻译结果递增地打印出来，不需要附加空间来存放子表达式的翻译结果



语法分析树中的各结点的属性值



实现翻译的语义动作

内容纲要

- 词法分析器的作用
- 词法单元的规约
- 词法单元的识别
- 词法分析器生成工具LEX
- 有穷自动机 (Finite Automata)
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

内容纲要

- 词法分析器的作用
  - 词法单元、模式、词素
- 词法单元的规约
  - 串和语言，正则表达式和正则定义
- 词法单元的识别
- 词法分析器生成工具LEX
- 有穷自动机 (Finite Automata)
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

词法分析

- 对源程序分析时，输入其实是一个字符串

```
if (i == j)
    z = 0;
else
    z = 1;
```

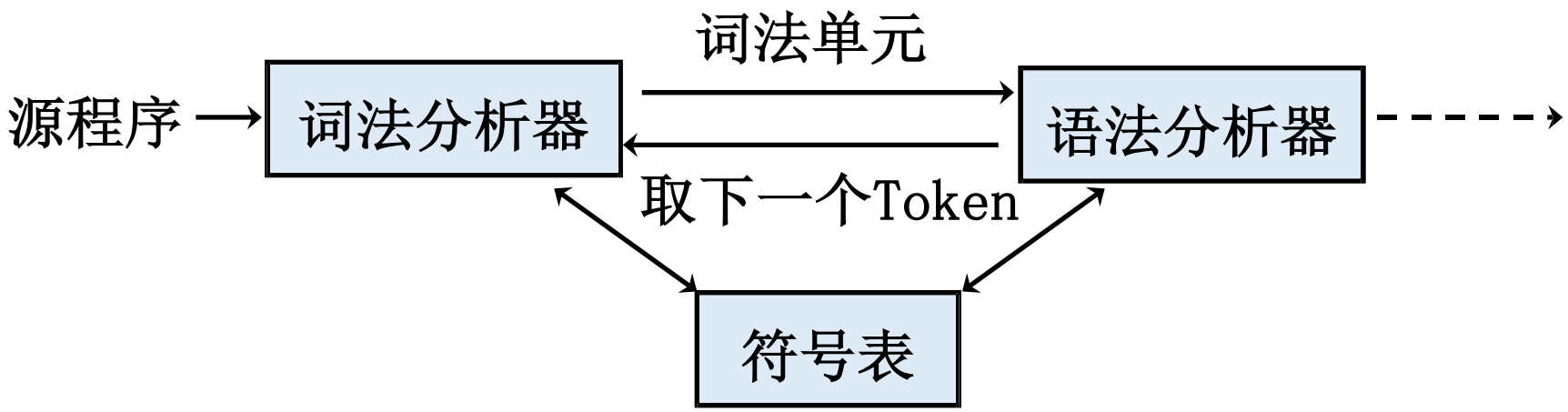
\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;

字符串中包含：制表符，空格，换行符，字母，数字，括号，赋值符号，标点等等

- 词法分析：读入源程序的输入字符串，将其组成词素，根据类型，生成并输出一个词法单元序列

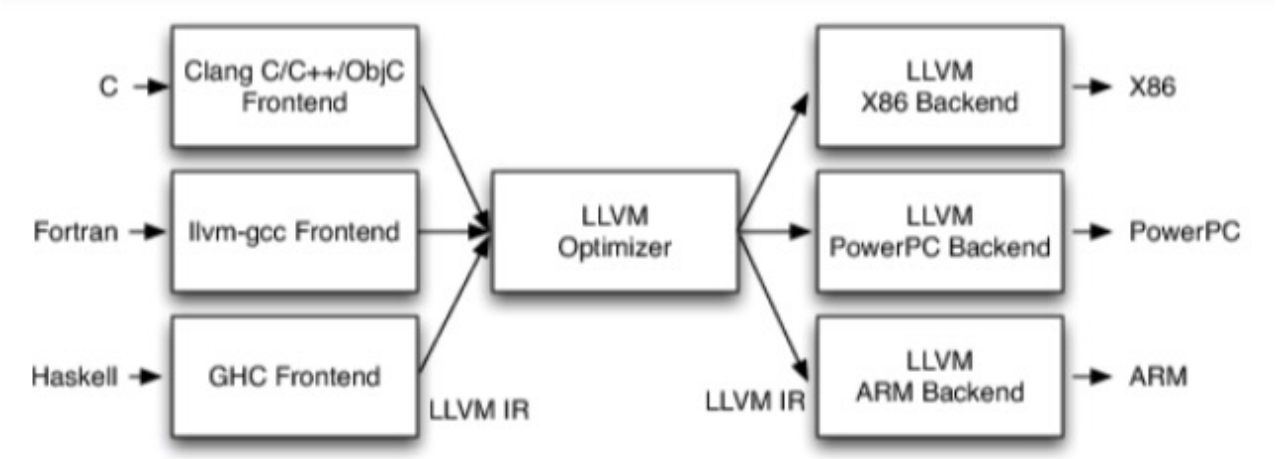
词法分析器的作用

- 读入源程序的输入字符，识别词素，输出词法单元（token）序列
- 与符号表交互，将词法单元信息添加到符号表中
- 过滤注释、空白、换行符、制表符等
- 记录换行符的个数，将编译器生成的错误消息与源程序的位置联系起来
- 预处理操作，比如识别宏、宏的扩展等



分析阶段

- 编译器的分析阶段划分为词法分析和语法分析
  - 简化编译器的设计：明确各部分的目标。
  - 提高编译器的效率：使用高效的词法分析算法。
  - 增强编译器的可移植性：输入字符方法与机器相关。



Low Level Virtual Machine

概念与术语

- 词法单元/记号（token）
  - <词法单元名，属性值（可选）>
  - 词法单元名是表示词法单元**种类**的**抽象符号**；语法分析器通过词法单元名即可确定词法单元序列的结构。
  - 属性值是相应的**附加信息**，通常用于语法分析之后的翻译阶段
- 模式（pattern）
  - 描述了**一类**词法单元的词素可能具有的**形式**
- 词素（lexeme）
  - 源程序中的字符序列
  - 它和某个词法单元的模式来**匹配**，被词法分析器识别为该词法单元的一个**实例**

三者之间的关系

- 在输入中有一组字符串会产生相同的记号（作为输出），这个字符串构成的集合由一个与该记号相关联的称为模式的规则来描述。
- 这个模式被说成匹配该集合中的每个字符串。

词法单元	模式的非正式描述	词素示例
if	字符 i，f	if
else	字符 e，l，s，e	else
comparison	< 或 > 或 <= 或 >= 或 == 或!=	<=，!=
id	字母开头的字母/数字串	pi，score，D2
number	任何数字常量	3.1415926，0，6.02e23
literal	在两个"之间的除引号以外的任何字符	“core dumped”

词法单元的类别

- 关键字（keyword）
  - begin, end, if, then, for, while
- 运算符（operator）
  - +, -, *, /, <, >=
- 标识符（identifier）
  - 表示各种名字
- 常量
  - 数字（number）：256， 3.1415926
  - 字面量（literal）：“abc”
- 分界符（delimiter）
  - 逗号、分号、冒号等

词法分析示例-1

- 读入源程序的字符流，过滤注释和空白，识别词素

```
while  i<>j  do
    if  i>j  then  i:=i-j  else  j:=j-i
```

“while”, “i”, “< >”, “j”, “do”, “if”, “i”, “>”, “j”, “then”, “i”, “:=”, “i”, “-”, “j”, “else”, “j”, “:=”, “j”, “-”, “i”

词法分析示例-1

- 与相应的模式匹配，输出一个词法单元序列

```
while  i<>j  do
    if  i>j  then  i:=i-j  else  j:=j-i
```

上述源程序经词法分析器的输出：

〈while，——〉
〈id，指向符号表中i的条目的指针〉
〈relational-op，NE〉
〈id，指向符号表中j的条目的指针〉
〈do，——〉
〈if，——〉
〈id，指向符号表中i的条目的指针〉
……
〈id，指向符号表中i的条目的指针〉

问题

- 如何将字符流分成不同的词法单元?
- 如何在扫描输入字符流时，决定一个词法单元的开始和结束?

词法分析示例-2

- FORTRAN语言
  - 空格是没有含义的，例如

    VAR1 等同于 VA R1

  - 对比如下两段代码：

```
DO 5 I = 1,25

DO 5 I = 1.25
```

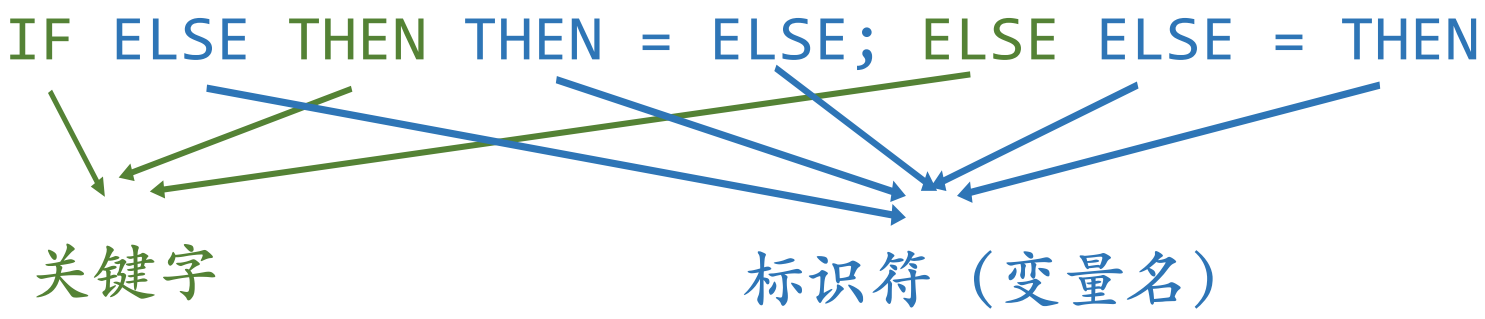
DO循环语句

赋值语句
DO5I = 1.25

- 在从左向右的分析过程中，有时候需要**向前看**(Lookahead)

词法分析示例-3

- PL/I语言
  - 关键字不是保留字（reserved word）



词法分析示例-3

- PL/I语言
  - 对于语句：

    DECLARE (ARG1,. . ., ARGN)

  - 无法决定DECLARE是用于声明的关键字还是数组名
    - 需要继续看前面的字符来决定
  - 在词法分析过程中，有时候需要**向前看任意多个字符**

词法分析示例-4

- C++语言
  - C++ template：

    ```
    Foo<Bar>
    ```

  - C++ 输入流：

    ```
    cin >> var;
    ```

  - 下面的语句如何分析?

    ```
    Foo<Bar<Barr>>
    ```

小结

- 词法分析的目标
  - 将输入字符流分成多个词素
  - 识别每个词素，生成相应的词法单元
- 在从左向右扫描字符流时，有时候需要向前看

内容纲要

- 词法分析器的作用
  - 词法单元、模式、词素
- **词法单元的规约**
  - **串和语言，正则表达式和正则定义**
- 词法单元的识别
- 词法分析器生成工具LEX
- 有穷自动机 (Finite Automata)
- 正则表达式到有穷自动机
- 词法分析器生成工具的设计

如何描述词素的模式?

- 我们需要一种方法
  - 可以描述不同类型词法单元的词素
  - 可以避免混淆
    - if 是否是两个变量 i，j?
    - ==是否是两个赋值符号=?

- 正则表达式是一种描述模式的重要方法
  - 简洁，高效，易理解

词法单元的规约

- 规约（Specification）
- 主要内容
  - 字母表
  - 符号串和语言
  - 正则表达式
  - 正则定义
  - 正则表达式的扩展

字母表（Alphabet）

- 字母表是一个有限的符号集合
  - 典型的符号包括字母、数字、标点符号等
- 任何程序语言都有自己的字母表
  - 机器语言的字母表：由 0 和 1 组成的二进制字母表$\Sigma=\{0, 1\}$
  - ASCII字符集
  - Pascal 语言的字母表：

    $\Sigma=\{A\sim Z, a\sim z, 0\sim 9, +, -, *, /, <, =, >, :, ‘, ’, ;, ., ^, (,), \{, \}, [,]\}$
- 可以类比自然语言的字母表

符号串（String）

- **符号串**：某个字母表上的**串**是该字母表中**符号的有穷序列**

- 定义：已知字母表Σ，
  1) ε 是Σ上的一个长度为0的符号串，称为空串；
  2) 若 α 是Σ上的符号串，而 a 是Σ的元素，则 αa 是Σ上的符号串；
  3) β 是Σ上的符号串，当且仅当它由(1)(2)导出（递归定义）。
  4) 串s的长度，通常记作$|s|$

通常约定

- 用英文字母表比较靠前小写英文字母表示**符号**
  - $a, b, c, d, \cdots,$
- 用小写希腊字母或字母表中靠后的英文字母表示**符号串**
  - $\alpha, \beta, \gamma, \delta, \theta, \pi, \rho$
  - $\cdots, x, y, z$
  - ε 通常用来表示空串
- 用大写英文表示**符号串集合**
  - $A, B, C, D, \cdots, X, Y, Z$

串的相关术语

- 已知符号串s
  - 串s的**前缀(prefix)**：删除 s 尾部的零个或多个连续的符号
  - 串s的**后缀(suffix)**：删除 s 头部的零个或多个连续的符号
  - 串s的**子串(substring)**：从 s 中删去某个前缀和某个后缀
  - 串s的**真前缀**、**真后缀**、**真子串**分别是s的既不等于ε，也不等于s本身的前缀、后缀、子串
  - 串s的**子序列(subsequence)**：从 s 中删去零个或多个符号(这些符号不要求连续)
  - **逆转(reverse, 或称转置，用s^R表示)**：将 s 中的符号按相反次序写出而得到的串
  - **长度(length)**：符号串中的符号的数目。例如, |aab|=3, |ε|=0。

串的示例

- 假设符号串 s=banana
  - 前缀：ε, b, ba, ban, bana, banan, banana
  - 后缀：banana, anana, nana, ana, na, a, ε
  - 子串：banana, anana, banan, anan, ⋯, ε
  - 真前缀，真后缀，真子串：y≠s & y≠ε
  - 子序列：baa (这些符号不要求是连续的)
  - 逆转 s^R：ananab
  - 长度：|banana|=6

串的运算

- 连接（concatenation）
  - 设 x 和 y 是符号串，它们的连接 xy 是把 y 的符号附加在 x 后形成的串
  - 例如：x=dog且y=house，那么xy=doghouse
  - 对于任何串 x 都有 $\varepsilon x = x\varepsilon = x$

- 指数（exponentiation）
  - 定义 x^0 为 ε，并且对于 i>0, $x^i = x^{i-1}x$
  - $x^0 = \varepsilon$, $x^1 = x$, $x^2 = xx$, $x^3 = xxx$, …
  - 例如：x=ba, x^1=ba, x^2=baba, x^3=bababa, …

语言（Language）

- 语言：某个给定字母表上一个任意的可数的符号串集合

- 语言的例子
  - 空集 ∅
  - 只包含空串的集合{ε}
  - 所有符合规范的C语言标识符的集合
  - 所有语法正确的C语言程序的集合
  - 所有语法正确的英语句子的集合

字母表	语言
英文字母	英语句子的集合
ASCII码	C程序的集合

语言的运算

- 设L和M是两个符号串集合，则:
  - 并(union)：$L \cup M = \{ s \mid s$ 属于 L 或 s 属于 M$\}$
  - 连接(concatenation)：$LM = \{ st \mid s$ 属于 L 且 t 属于 M$\}$
  - 指数(exponentiation)：$L^0 = \{\varepsilon\}$，$L^1 = L$, $L^2 = LL$, $\cdots$, $L^n = L^{n-1}L$
  - 语言L的Kleene闭包(closure)，记作L^*： $L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \ldots$
  - 语言L的正闭包(positive closure)，记作L^+: $L^+ = \bigcup_{i=1}^{\infty} L^i = L^1 \cup L^2 \cup L^3 \cup \ldots$

语言运算示例

- 令 L 表示字母的集合{A, B, C, ⋯, Z, a, b, c,⋯, z}, D表示数字的集合{0, 1, 2, ⋯, 9}，可以通过运算构造出新的语言：

  1) $L \cup D = \{A \sim Z, a \sim z, 0 \sim 9\}$

  2) LD 是所有由一个字母后跟一个数字组成的符号串的集合

  3) L^4 是所有由四个字母构成的符号串的集合

  4) L^* 是所有由字母构成的符号串的集合

  5) $L(L \cup D)^*$ 是所有以字母开头的，由字母和数字组成的符号串的集合

  6) D^+是由一个或多个数字构成的串的集合。

Quiz：判断题

- $L \cup M = M \cup L$?
- $LM = ML$?
- $\{\varepsilon\}M = M\{\varepsilon\}$?

正则表达式（Regular Expression）

- **正则表达式**是一种表示方法
  - 可以**描述**所有通过对某个字母表上的符号应用运算符而得到的语言
- 例如：
  - 已知集合 L ={A, B, C, ⋯, Z, a, b, c,⋯, z}, D={0, 1, 2, ⋯, 9}，则 L(L∪D)* 是所有以字母开头的，由字母和数字组成的符号串的集合
  - C语言中的合法**标识符**具有类似的定义，可以使用如下**正则表达式**来描述对应于C语言标识符的语言：

$$letter_\ (\ letter_\ |\ digit\ )\ *$$

注： *letter_*代表任一字母或者下划线， *digit*代表数字， | 代表并运算， *代表Kleene闭包运算，并列放置表示连接运算

正则表达式

- 以**递归**的方式定义某个字母表∑上的正则表达式，以及其所表示的正则集合（正则语言）
  - ε 是一个正则表达式，则 L(ε) ={ε}，即该语言只包含空串
  - 如果 a 是∑上的一个符号，则a是一个正则表达式，且 L(a) = {a}
  - 归纳步骤：设 r 和 s 是∑上的正则表达式
    - ❖ (r)|(s)是一个正则表达式，表示语言 L(r)∪L(s)
    - ❖ (r)(s)是一个正则表达式，表示语言 L(r) L(s)
    - ❖ (r)*是一个正则表达式，表示语言 (L(r))*
    - ❖ (r)是一个正则表达式，表示语言 L(r)
- 根据运算符的优先级，去掉冗余括号: (a)|((b)*(c)) = a|b*c

正则表达式示例-1

- 令字母表$\sum=\{a, b\}$
  1) 正则表达式 a|b 表示语言 $\{a, b\}$
  2) 正则表达式 (a|b)(a|b) 表示语言$\{aa, ab, ba, bb\}$
  3) 正则表达式 a* 表示语言 $\{\varepsilon, a, aa, aaa, \cdots\}$
  4) 正则表达式 (a|b)* 表示语言 $\{\varepsilon, a, b, aa, ab, ba, bb, aaa, \cdots\}$
  5) 正则表达式 a|a*b 表示语言 $\{a, b, ab, aab, aaab, aaaab\cdots\}$

正则表达式示例-2

- C语言标识符：

$$(A|B|\ldots|Z|a|b|\ldots|z|_)((A|B|\ldots|Z|a|b|\ldots|z|_)|(0|1|\ldots|9))^*$$

- 有符号整数:

$$(+|-|\varepsilon)(0|1|\ldots|9)(0|1|\ldots|9)^*$$

Quiz：选择题

下面的正则表达式对应的语言中包含多少个不同字符串？

$$(0|1|\varepsilon)(0|1|\varepsilon)(0|1|\varepsilon)(0|1|\varepsilon)$$

a) 12
b) 16
c) 31
d) 32
e) 64
f) 81

正则表达式的代数定律

- 设 r、s 和 t 是某字母表∑上的正则表达式，其遵守的代数定律有：
  - 交换律：$r|s = s|r$
  - 结合律：$r|(s|t) = (r|s)|t$　　$r(st) = (rs)t$
  - 分配律：$r(s|t) = rs|rt$　　$(s|t)r = sr|tr$
  - ε是连接的单位元：$\varepsilon r = r\varepsilon = r$
  - 闭包中一定包含ε：$r^* = (r|\varepsilon)^*$
  - 幂等性：$r^{**} = r^*$
  - 此外：$(r|s)^* = (r^*s^*)^*$

正则表达式示例

- 令字母表$\Sigma=\{a, b\}$
  1) 正则表达式 a|b 表示语言 $\{a, b\}$
  2) 正则表达式 (a|b)(a|b) 表示语言$\{aa, ab, ba, bb\}$
  3) 正则表达式 a* 表示语言 $\{\varepsilon, a, aa, aaa, \cdots\}$
  4) 正则表达式 (a|b)* 表示语言 $\{\varepsilon, a, b, aa, ab, ba, bb, aaa, \cdots\}$
  5) 正则表达式 a|a*b 表示语言 $\{a, b, ab, aab, aaab, aaaab\cdots\}$

能否简化？

- C语言标识符：

  (A|B|...|Z|a|b|...|z|_) ((A|B|...|Z|a|b|...|z|_) | (0|1|...|9))*

- 有符号整数:

  (+|-|ε) (0|1|...|9) (0|1|...|9)*

正则定义（Regular Definition）

- 为表示方便，我们希望为某些正则表达式**命名**，并在之后的正则表达式中像使用符号一样使用这些名字
- **正则定义**具有如下形式的定义序列：

$$
\begin{aligned}
D_1 &\rightarrow R_1 \\
D_2 &\rightarrow R_2 \\
&\vdots \\
&\vdots \\
D_n &\rightarrow R_n
\end{aligned}
$$

限定：

- 每个 D_i 是新符号，不在字母表中且各不相同
- 每个 R_i 中只能出现字母表中的字符，或者前面已定义的正则表达式名字

其中，$R_1, R_2, \ldots\ldots, R_n$ 为正则表达式，$D_1, D_2, \ldots\ldots, D_n$ 为正则表达式名字

正则定义示例-1

- C语言标识符的正则定义

$$letter_ \rightarrow A \mid B \mid \cdots \mid Z \mid a \mid b \mid c \cdots \mid z \mid _$$

$$digit \rightarrow 0 \mid 1 \mid \ldots \mid 9$$

$$id \rightarrow letter_ \; (letter_ \mid digit) *$$

正则定义示例-2

- 整数：不能为空的数字串

$$digit \rightarrow 0 \mid 1 \mid \ldots \mid 9$$

$$digits \rightarrow digit\ digit^*$$

正则定义示例-3

- 无符号数：整数，浮点数
  - 5280，3.1415，6.33E4，1.89E-4

$$digit \rightarrow 0 \mid 1 \mid \ldots \mid 9$$

$$digits \rightarrow digit\ digit^*$$

$$optionalFraction \rightarrow .\ digits \mid \varepsilon$$

$$optionalExponent \rightarrow (\ E\ (\ + \mid - \mid \varepsilon\ )\ digits\ ) \mid \varepsilon$$

$$number \rightarrow digits\ optionalFraction\ optionalExponent$$

正则表达式的扩展形式

- 为了表达方便，通常可以对正则表达式做如下的扩展：
  - 一次或多次出现：$(\mathbf{r})^+$ 用来表示语言 $(\mathbf{L}(\mathbf{r}))^+$，表示正则表达式及其语言的正闭包
    - $\mathbf{r}^* = \mathbf{r}^+ | \varepsilon$;　　$\mathbf{r}^+ = \mathbf{r}\mathbf{r}^* = \mathbf{r}^*\mathbf{r}$
    - 与闭包运算*具有同样的优先级
  - 零次或一次出现：r? 用来表示 $\mathbf{r}|\varepsilon$，其表示语言 $\mathbf{L}(\mathbf{r}?) = \mathbf{L}(\mathbf{r}) \cup \{\varepsilon\}$
  - 字符类：一个正则表达式 $a_1|a_2|\ldots|a_n$ 可以缩写为 $[a_1a_2\ldots a_n]$，而连续的序列 $[a_1a_2\ldots a_n]$ 可以缩写为 $[a_1 - a_n]$
    - **[abc]** 表示 **a|b|c**　　**[a-z]** 表示 **a|b|c|…|z**

扩展形式示例-1

- 基于正则表达式的缩写表示法，可以改写正则定义:

C语言标识符

$letter_ \rightarrow A \mid B \mid \cdots \mid Z \mid a \mid b \mid c \cdots \mid z \mid _$

$digit \rightarrow 0 \mid 1 \mid \ldots \mid 9$

$id \rightarrow letter_\ (\ letter_ \mid digit\ )\ *$

$\Rightarrow$

$letter_ \rightarrow [A\text{-}Za\text{-}z_]$

$digit \rightarrow [0\text{-}9]$

$id \rightarrow letter_\ (\ letter_ \mid digit\ )\ *$

扩展形式示例-2

- 基于正则表达式的缩写表示法，可以改写正则定义：

无符号数

digit → 0 | 1 | ... | 9
digits → *digit digit**
optionalFraction → . *digits* | ε
optionalExponent → (E (+ | - | ε) *digits*) | ε
number → *digits optionalFraction optionalExponent*

digit → [0-9]
digits → *digit*+
number → *digits* (. *digits*)? (E [+-]? *digits*)?

Lex中的正则表达式

表达式	匹配符号	举例
c	单个非运算符字符 c	a
\c	转义符：字符c的字面值(literal value)	*
"s"	串s的字面值	"**"
.	除换行符之外的任何字符	a.*b
^	一行的开始	^abc
$	一行的结尾	abc$
[s]	字符类（字符串s中的任一个字符）	[abc]
[^s]	补集字符类（不在串s中的任一个字符）	[^abc]
r{m, n}	最少m个，最多n个r的重复出现	a{1,5}
r_1 / r_2	后面跟着r_2的r_1	abc/123
与之前讲的一样：r*, r+, r? (重复） r_1 r_2 （连接） r_1 | r_2 （或）		